Set up your own RocketWire

Step-by-step: your own copy of the site, with your own Supabase database and accounts.

> **Why this keeps accounts separate:** Supabase accounts and project data live inside one Supabase *project*. If your team creates its own Supabase project and points its own copy of the site at it, nobody on your team can see or touch the original team's accounts or data, and the reverse is also true. The two sites share nothing but the code.

What you need

- A free **GitHub** account (to hold and host your copy of the code)
- A free **Supabase** account (supabase.com)
- About 20 minutes. No build tools: RocketWire is plain HTML and JavaScript.

Step 1. Get your own copy of the code

1. Ask the RocketWire owner for access to the repository, or for a zip of the folder.
2. On GitHub, create a **new repository** in your own account (e.g. `myteam-rocketwire`). Make it **Private** if you can (see the security note at the end).
3. Upload the files into it (or clone the original and push to your new repo). Do not keep a link to the original repo's Supabase settings; you will replace them in Step 4.

Step 2. Create your Supabase project

1. Go to **supabase.com** and sign up, then click **New project**.
2. Pick a name (e.g. *myteam-rocketwire*), set a strong **database password** (save it in a password manager), and choose the region closest to your team.
3. Wait about a minute for it to finish setting up.

Step 3. Create the database table

RocketWire stores everything in one table called `projects`. In Supabase open **SQL Editor** → **New query**, paste the whole block below, and click **Run**.

```
create table public.projects (
  id          text primary key,
  user_id     uuid not null references auth.users(id) on delete cascade,
  name        text,
  data        jsonb,
  updated_at  timestamptz not null default now()
);

create index projects_user_id_idx on public.projects(user_id);

-- Row-level security: each signed-in user can only see and change their own rows
alter table public.projects enable row level security;

create policy "own rows: select" on public.projects
  for select using (auth.uid() = user_id);
create policy "own rows: insert" on public.projects
  for insert with check (auth.uid() = user_id);
create policy "own rows: update" on public.projects
  for update using (auth.uid() = user_id) with check (auth.uid() = user_id);
create policy "own rows: delete" on public.projects
  for delete using (auth.uid() = user_id);
```

> **Do not skip row-level security.** The Supabase key in the website code is public by design. Row-level security is what stops one user from reading another user's projects.

This schema was worked out from how the code reads and writes data (columns `id, user_id, name, data, updated_at`), not copied from the original database. If you see an error when pushing, check that these column names match.

Step 4. Connect the code to your Supabase

1. In Supabase go to **Project Settings** → **API Keys** (older dashboards: **API**).
2. Copy the **Project URL** and the **publishable** key (starts with `sb_publishable_`; on older projects it is the **anon public** key). **Never** use the `secret` or `service_role` key in the website.
3. Open `js/supabase.js` and replace lines 4 and 5:

```
const SUPABASE_URL = 'https://YOUR-PROJECT-ID.supabase.co';
const SUPABASE_KEY = 'sb_publishable_YOUR_KEY_HERE';
```

This is the only change required for the site to talk to your database instead of the original one.

Step 5. Create the shared team account

> **Important: projects belong to one account.** RocketWire saves every project under the account that is signed in, and each account can only see its own projects. If every teammate made their own account, they would **not** see each other's work. So for collaboration, your whole team signs in to **one shared account**.

5a. Create the account (admin, once)

1. In Supabase open **Authentication** → **Users** → **Add user** → **Create new user**.
2. Enter an email for the team (e.g. *team@yourcompany.com*, or any address you control) and a **strong, unique password that you choose**. This is where the password is set. It is not written in any file.
3. Tick **Auto Confirm User** so no confirmation email is needed, then click **Create user**.
4. Share the email and password with your teammates privately (a password manager, not a public chat).

Two ways to create the account: dashboard or website button

Both do the same thing underneath: they add one user to **Authentication** → **Users** in Supabase. Once the user exists, logging in is identical and projects belong to whichever user is signed in. You only need **one** of these routes.

	Supabase dashboard (5a, recommended)	**"Create account" on the website**
Who does it	The admin	Anyone who opens the site
Email confirmation	Skipped if you tick *Auto Confirm User*	Required by default: Supabase emails a link and you cannot log in until you click it
Password	The admin types it	Typed in the login window
Result	A normal Supabase user	The same kind of user

Why the dashboard is recommended for a shared account: it skips the email step and Supabase's low email limit. Also, if you turn off **Allow new users to sign up** (5d), the website's "Create account" button stops working, so the dashboard becomes the only way to create the account. If the team account already exists, nobody needs "Create account" again; teammates just click **Log in**.

5b. Everyone logs in

1. Open the RocketWire site and click **Log in**.
2. Type the team **email** in the Username box and the team **password** in the Password box. Click **Log in** (not "Create account").
3. The email appears at the top and the **Push** button shows, which means you are connected to Supabase.

5c. How teammates share work

- **Push** sends your changes to Supabase. **Sync** pulls what teammates pushed. The site also pulls automatically when you return to the tab.
- Click **Push** when you finish a change and **Sync** before you start, so you see the latest version.
- The app merges changes made on different devices, but if two people edit the same item at the same moment, one edit can overwrite the other. Agree who edits what, and push often.
- Everyone appears as the same user, so there is no record of who made which change.

5d. Lock down sign-ups (recommended)

In Supabase open **Authentication → Sign In / Providers** (or **Settings**) and turn **off "Allow new users to sign up"**. The site still shows a "Create account" button, but strangers can no longer make accounts on your database.

The hardcoded login in constants.js

`js/constants.js` line 4 contains an offline-only fallback login:

```
const CREDS = {user:'rocketteam',pass:'launch2026'};
const TEAM_SUPABASE_EMAIL = '';
```

- It never touches your Supabase account. It only lets someone into the site locally, with no syncing.
- Because it is in the code, anyone who opens the site can read it. **Change `pass` to a long random value** (or leave it, it only gives local access and no cloud data).
- You do **not** need `TEAM_SUPABASE_EMAIL`. Leave it empty and log in with the shared email in 5b. That keeps the real password out of the code. (If you did put the team email there, the username *rocketteam* would sign in as that account, but the Supabase password would then have to match `CREDS.pass` and be visible in the code. Not recommended.)

Alternative: separate accounts per person

Use this only if people should have **private**, separate projects. Each person types their email and chosen password into the login window, clicks **Create account**, clicks the confirmation link Supabase emails them (it opens your site), then logs in with the same email and password. Their projects are visible only to them. There is no site button for forgotten passwords; the admin resets them in **Authentication** → **Users**.

Step 6. Put the site online

1. In your GitHub repo go to **Settings** → **Pages**.
2. Source: **Deploy from a branch**, branch **main**, folder **/ (root)**. Save.
3. After a minute your site is at `https://YOURNAME.github.io/REPO-NAME/`. (Private repos need a paid GitHub plan for Pages; free alternatives are Netlify or Cloudflare Pages, which also just serve the folder as-is.)
4. Back in Supabase open **Authentication** → **URL Configuration**. Set **Site URL** to your site address, and add it under **Redirect URLs**. Without this, confirmation emails will link to the wrong place.

Step 7. Test that it works

1. Open your site and log in with the shared team email and password from Step 5.
2. Create a project, add a system, then click **Push**.
3. In Supabase open **Table Editor** → **projects**. Your project should appear as a row, with your user id in `user_id`.
4. Open the site in a different browser or device, log in with the same account, and click **Sync**. The project should appear.
5. Optional: a separate personal account (see the alternative in Step 5) should **not** see the team's projects. That proves row-level security is working.

Checklist: files you changed

File	Change	Required?
`js/supabase.js`	Your Supabase URL and publishable key	Yes
`js/constants.js`	New random `CREDS` password (leave `TEAM_SUPABASE_EMAIL` empty)	Advised
Supabase SQL Editor	Create `projects` table + policies	Yes

Supabase URL Configuration	Site URL + Redirect URL	Yes, if using email sign-up

Troubleshooting

- **"row-level security" error on Push:** the policies in Step 3 are missing, or you are signed in with the offline `CREDS` login instead of the team email (it can't push).
- **"Sync needs a Supabase account":** you logged in with the offline `CREDS` login. Log out and log in with the shared team email and password instead.
- **Sign-up email never arrives:** turn off Confirm email, or add your own SMTP provider.
- **Site still shows the old team's data:** your browser cached old files, or `supabase.js` still has the original URL. Hard-refresh (Cmd+Shift+R) and re-check Step 4.

Security notes

- Never put the Supabase `secret` / `service_role` key or your database password in any file of the site.
- Anything in the repo's JavaScript is visible to every visitor of the site, even if the GitHub repo is private.
- Back up your data occasionally: Supabase **Table Editor** → **projects** → **Export**, or use RocketWire's own export.